

Lebanese American University



COE 546 - Machine Learning

Dr. Fouad Trad

User Behavior Prediction in Food Delivery Applications

Lara Snih

Jana Antoun

Ghadi Ammar

April 27th, 2026

Table of Contents

Introduction.....	5
Problem Understanding	5
Exploratory Data Analysis (EDA)	6
Dataset.....	6
Target Distribution.....	6
Features	7
i. Numerical	7
ii. Categorical.....	8
iii. Temporal.....	9
iv. Missing Data.....	9
Data Preprocessing.....	10
i. Handling of Missing Data	10
Feature Engineering.....	12
Modeling Approach	14
1. Random Forest.....	14
2. Moving to Gradient Boosting: XGBoost, LightGBM, and CatBoost.....	14
i. XGBoost (Extreme Gradient Boosting).....	14
ii. LightGBM (Light Gradient Boosting Machine).....	14
iii. CatBoost (Categorical Boosting).....	14
Feature and Hyperparameter Tuning	15
Ensemble.....	15
Evaluation	16
Metric.....	16
Validation Strategy	16
Results.....	16
Kaggle Score	16
Discussion.....	17
Conclusion	18

Table of Figures

Figure 1 - Dataset information.....	6
Figure 2 - Target distribution.....	6
Figure 3 - Feature description.....	7
Figure 4 - Distribution of numerical features (median) by target class	8
Figure 5 - Numerical feature distribution (logarithmic)	8
Figure 6 - Categorical features conversion rates.....	8
Figure 7 - Temporal feature conversion rates	9

Table of Tables

Table 1 - Feature categorization	7
Table 2 - Summary of data preprocessing	11
Table 3 - Most important engineered features	13
Table 4 - Summary of model progression.....	15

Introduction

The goal of this project is to build a binary classifier that predicts whether a user will place a food delivery order given a set of anonymized session-level signals. The model's primary evaluation metric is the Area Under the ROC Curve (AUC), which is appropriate for imbalanced binary classification tasks such as this one. To address this problem, we follow a full machine learning pipeline including exploratory data analysis, preprocessing, feature engineering, model building, evaluation, and ensemble tuning in order to explore and compare multiple models. The focus is on delivering strong AUC performance while uncovering behavioral drivers of order completion.

The objective of this project is to predict whether a user will complete an order during a food delivery session based on behavioral, transactional, temporal, and promotional data. This is a supervised binary classification problem where the target variable (`order_placed`) indicates whether an order was completed (1) or not (0).

Each observation represents the final state of a user session and contains information such as session timing, cart activity, customer type, promotional interactions, device characteristics, and user responses to offers. The challenge lies in learning patterns from these diverse signals that distinguish sessions likely to convert from those that end without an order.

User behavior is often nonlinear and influenced by interactions among multiple variables rather than isolated features. Promotional effectiveness may depend on cart value or customer type, temporal behavior may reveal purchasing intent, and some features may carry hidden predictive signals only when combined through feature engineering. Because of these challenges, success depends not only on model selection but also on thoughtful preprocessing, engineered features, robust validation, and careful optimization.

Problem Understanding

At the beginning of the project, several important challenges became clear. The most significant was the severe class imbalance, where only 2.9% of sessions result in an order, creating a strong skew between the two classes. This makes accuracy unreliable and pushes the focus toward more robust evaluation methods like AUC. Another difficulty was that all features are anonymized (f2–f17), which meant there was no domain information to rely on, and each variable had to be understood from its statistical behavior.

In addition, a large portion of the data contains structured missing values, with f12, f13, f14, f15, and f17 missing. This suggested that missingness itself carries information rather than being random noise. The dataset also includes a mix of data types (numerical, categorical, and timestamp variables) each requiring different preprocessing approaches. Finally, with a relatively large training set and a separate unseen test set, a key requirement was ensuring that the model generalizes well without overfitting, especially given the imbalance in the target distribution.

Exploratory Data Analysis (EDA)

Exploratory data analysis was conducted to understand the structure of the dataset, identify important patterns, and guide preprocessing and modeling decisions. We focused on examining feature types, missing values, distributions, and relationships with the target variable. The dataset contained a mix of numerical, categorical, and timestamp-based or temporal variables, suggesting the need for multiple preprocessing strategies.

Dataset

The training set contains 297,236 rows and 18 columns (17 features and 1 target). The test set contains 99,639 rows and 17 columns. Below is a summary of column types:

```
Train shape : (297236, 18)
Test shape  : (99639, 17)

Column dtypes:
id          int64
f2          object
f3          datetime64[ns, UTC]
f4          datetime64[ns, UTC]
f5          datetime64[ns, UTC]
f6          int64
f7          object
f8          int64
f9          object
f10         int64
f11         float64
f12         object
f13         float64
f14         float64
f15         float64
f16         float64
f17         object
order_placed int64
dtype: object
```

Figure 1 - Dataset information

Target Distribution

The target variable is highly imbalanced where about 97.1% of the target belongs to class 0 (no order placed) and only 2.9% belong to class 1 (order placed). This severe imbalance plays a significant role on the model choice.

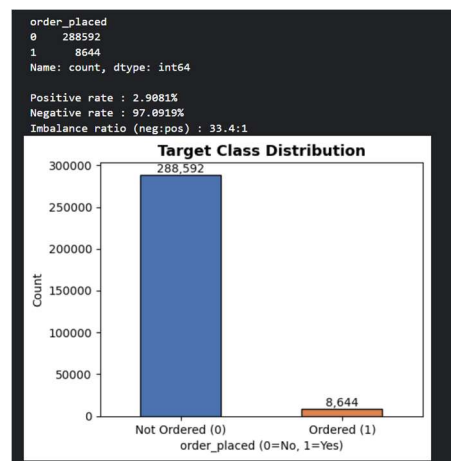


Figure 2 - Target distribution

Features

A crucial and essential step in designing a model is dealing with the features of the dataset by identifying their types, relevance, and the means to feature engineer or select and extract the features in the most effective manner.

Feature	Description
id	Unique Customer identifier
f2	Unique Session identifier
f3	Session start timestamp
f4	Session end timestamp
f5	Timestamp of the last recorded activity in the session
f6	Time zone associated with the session
f7	Type of action performed by the user (e.g., browsing, interaction)
f8	Number of promotional offers the user declined
f9	Customer type (new or returning)
f10	Number of items added to the cart during the session
f11	Total monetary value of items in the cart
f12	Promotion type: F (fixed discount with minimum spend), P (item-based discount), S (spend-more to unlock discount), C (checkout to unlock discount)
f13	Discount value associated with the promotion
f14	Minimum order value required to qualify for the promotion in f13
f15	Number of promotions displayed during the session
f16	Device screen size used during the session
f17	User response to the promotion (accepted, declined, ignored)
order_placed	Target variable indicating whether an order was completed (1 = yes, 0 = no)

Figure 3 - Feature description

Table 1 - Feature categorization

Category	Features
Numerical (continuous/integer)	f6, f8, f10, f11, f13, f14, f15, f16
Categorical	f7, f9, f12, f17
Temporal	f3, f4, f5
Identifier / dropped	id, f2

Observing the dataset¹, the following was revealed:

- i. *Numerical*
 - f10 showed a median of 0 for non-orders and 1 for orders placed which reveals the relation that users in their cart are more likely to place an order.
 - f11 showed a median of 0 for non-orders and 120 for orders which shows that a non-zero value indicates a higher likelihood to place an order.

¹ On Kaggle, we observed the train set and distribution of each feature through the dataset description (data explorer) tab in addition to some pyplot commands.

ii. *Categorical*

- f7 shows that PAGE_LOAD events exhibit a conversion rate of 2.97%, whereas CART_CHANGE events convert at only 0.10%. This inverse relationship suggests that CART_CHANGE may be associated with hesitation or abandonment behavior rather than completing an order.
- f9 observes a clear distinction between customer segments. OC users (interpreted as returning customers) convert at 4.18%, compared to 1.20% for NC users (new customers), indicating significantly stronger conversion propensity among returning users.
- f12 shows strong predictive behavior. Category S achieves a conversion rate of 37.2%, followed by C at 9.8%, while categories F and P remain below 2%. This indicates that f12 is one of the most informative categorical signals in the dataset.

order_placed	0	1
f6	300.0	300.0
f8	0.0	0.0
f10	0.0	1.0
f11	0.0	120.0
f13	10.0	10.0
f14	50.0	90.0
f15	1.0	1.0
f16	2529280.0	2401426.5

Figure 4 - Distribution of numerical features (median) by target class

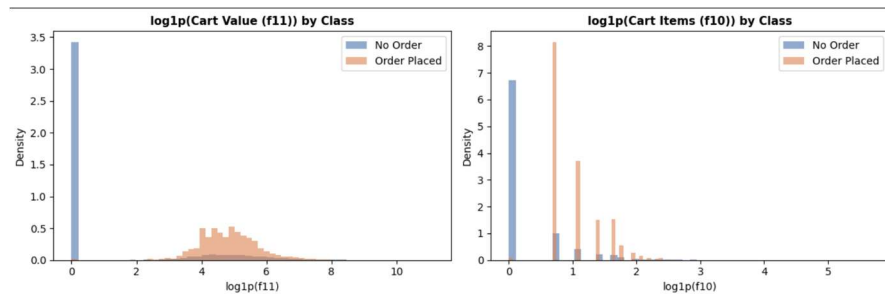


Figure 5 - Numerical feature distribution (logarithmic)

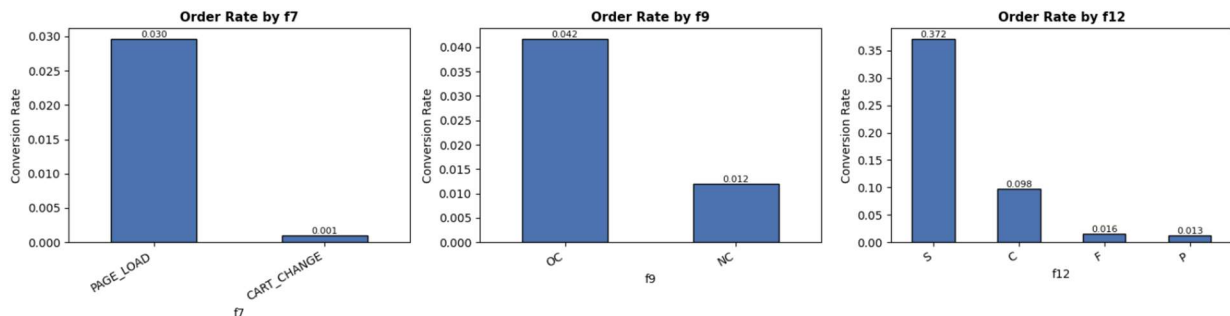


Figure 6 - Categorical features conversion rates

iii. Temporal

The timestamp-based features were converted and parsed into the following derived features which will be further explored in order to work with such temporal features more efficiently and determine patterns.

- Hour of day:
Conversion peaks during late-night hours (approximately 00:00–01:00), followed by a secondary increase in the evening (17:00–18:00). A noticeable trough is observed in the morning hours (09:00–10:00). This pattern is consistent with typical food ordering behavior driven by late-night and after-work demand.
- Day of week:
Strong weekly seasonality can be observed. Time differences between timestamp pairs (f4 - f3, f5 - f3) in seconds to capture session duration.

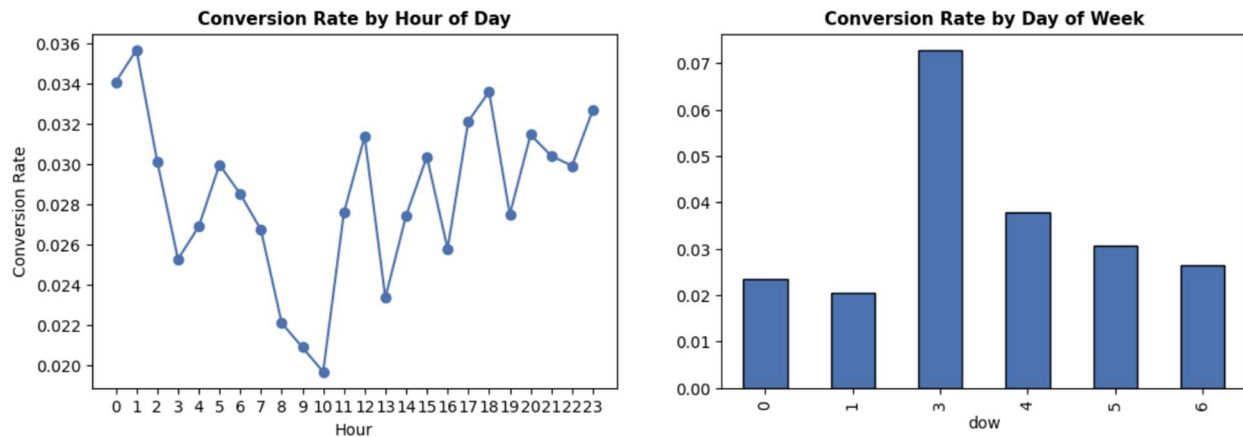


Figure 7 - Temporal feature conversion rates

iv. Missing Data

Five features (f12, f13, f14, f15, and f17) are jointly absent in training records, representing approximately 30.45% of the dataset. Critically, this missingness is not random. The five features are missing together as a block, suggesting they relate to a single underlying mechanism, such as whether an offer or promotion was active during that session. When no offer is present, all offer-related fields are empty simultaneously.

Data Preprocessing

i. Handling of Missing Data

As observed during EDA, missing values in features f12–f15 and f17 are not random but there is a pattern behind it. Instead of treating missing values as noise, the following strategy was applied:

- Categorical features (f7, f9, f12): missing values were explicitly encoded as a separate category (“NONE”), allowing models to learn missingness as a meaningful state.
- Numerical offer-related features (f13, f14, f15): missing values were retained and handled within feature engineering rather than aggressively imputed, since their absence itself carries behavioral meaning.

Importantly, no rows were removed, as EDA indicated that missingness is structurally informative rather than data corruption.

ii. Data Processing Temporal Features

As identified in EDA, temporal variables contain strong behavioral signals, particularly daily and weekly patterns in conversion behavior.

Instead of using raw timestamps (f3, f4, f5), they were transformed into engineered representations:

- Session duration and interaction timing were derived from timestamp differences.
- Time-based behavior was encoded using cyclical transformations to preserve periodic structure (hour and day-of-week patterns).

These transformations ensure that temporal relationships observed in EDA (such as night-time and mid-week peaks) are directly represented in model inputs. After transformation, the raw timestamp columns were removed to avoid redundancy and improve model stability.

iii. Categorical Variables

Categorical variables (f7, f9, f12) were retained due to their strong predictive power observed in the EDA. Missing values were replaced with a unified “NONE” category to maintain consistency across training and test sets.

No manual encoding such as one-hot encoding was applied. As we will reveal later due to our model choices another encoding method called model-native handling was used:

- CatBoost handled categorical variables internally using ordered target statistics.
- LightGBM processed categorical features via native categorical type conversion.

iv. Feature Transformation

Based on insights from EDA, additional transformations were applied to reduce skewness, normalize distributions, and capture behavioral intensity:

- Log transformations were applied to highly skewed numerical features such as cart-related variables.
- Ratio-based features were introduced to normalize cart behavior across different scales.
- Binary indicators were created to capture key behavioral thresholds (presence of items or cart value).

v. Feature Exclusion

Certain features were excluded to ensure model validity and prevent leakage:

- Identifier fields (id, f2) were removed as they contain no predictive information.
- Raw timestamp columns (f3, f4, f5) were dropped after feature extraction.
- f17 was excluded due to its conditional availability depending on session context, which introduces potential leakage and missingness bias.
- The target variable (order_placed) was excluded from input features by definition.

Table 2 - Summary of data preprocessing

Decision Point	Chosen Approach
Missing categoricals	“NONE” imputation
Missing numericals	Median imputation
Temporal features	Duration extraction + cyclical encoding
Categorical encoding	Native tree-based (CatBoost/LightGBM)
Row deletion	None
Exclusion	f17, id, f2 and raw timestamps all dropped

Feature Engineering

After initial preprocessing, the next step focused on building features that better represent user behavior during a session, rather than relying solely on the raw anonymized variables. The main idea was to translate patterns observed in the EDA into features that the model can directly learn from.

A first important aspect was handling missing data more intentionally. A binary feature was created to indicate when f12, f13, f14, f15, and f17 are all missing together. Since this missingness appears in a structured way rather than randomly, it likely reflects a specific type of session or event. Encoding it explicitly allows the model to treat “missingness” as a meaningful signal instead of noise.

Cart behavior was another key focus. Two simple binary indicators were introduced which are whether the cart contains any items (has_items) and whether it has any monetary value (has_cart_value). These were motivated by a clear pattern in the data where most non-converting sessions have zero items and zero cart value. In addition, a normalized feature (cart_value_per_item) was added to capture the average value per item, making cart value more comparable across different cart sizes.

Time-related behavior was also incorporated. Hour of day and day of week were extracted from the session timestamp (f3), since EDA showed strong daily and weekly cycles in conversion behavior. These features help the model capture predictable patterns such as higher activity in the evening and noticeable variation across weekdays.

To capture how users progress through a session, time differences between key events were computed (delta_f4_f3 and delta_f5_f3). These features act as simple indicators for how quickly or slowly a user moves through the funnel which is associated with their likelihood to complete a purchase.

Finally, interaction features were introduced to capture relationships between variables that may not be visible individually. The most important interaction combines customer type (f9) with session category (f12), allowing the model to learn how different user groups behave under different conditions.

This table summarizes the most impactful feature groups used in most of our models after iterative experimentation:

Table 3 - Most important engineered features

Feature	Description	
session_duration_s	f4 – f3 (seconds)	Captures overall session length as an indicator for engagement depth
time_to_event_s	f5 – f3 (seconds)	Measures how quickly the user reaches a key action
hour_sin / hour_cos	Cyclical encoding of hour	Preserves daily ordering patterns without discontinuity
dow_sin / dow_cos	Cyclical encoding of day of week	Captures weekly behavioral cycles and demand shifts
log_cart_value	$\log(1 + \text{cart value})$	Reduces skewness in spending behavior
log_cart_items	$\log(1 + \text{number of items})$	Stabilizes variability in cart size
avg_item_value	cart value per item	Normalizes spending intensity across carts
has_items	binary indicator for cart presence	Captures basic purchase intent
cart_value_rate	cart value per unit time	Approximates purchase urgency
has_offer	binary indicator of promotion exposure	Captures whether incentives were present
discount_ratio	discount relative to cart value	Measures strength of promotion impact
eligible_for_offer	cart meets threshold condition	Captures likelihood of reward activation
threshold_gap	distance from offer requirement	Measures how close user is to qualification
pct_to_threshold	progress toward eligibility	Normalized measure of purchase progression
offer_generosity	discount relative to threshold	Reflects attractiveness of promotion
is_fast_action	quick decision indicator	Captures high-intent or impulsive behavior
is_returning	returning vs new customer flag	Distinguishes user experience level
returning_with_cart	returning user $\times$ cart activity	Captures strong intent in experienced users
returning_with_offer	returning user $\times$ promotion exposure	Models promo sensitivity among loyal users
promo_acceptance_rate	historical promo response rate	Estimates user responsiveness to promotions
missing_block	structured missingness indicator	Captures missing data as behavioral signal

Modeling Approach

1. Random Forest

The first model attempted was a Random Forest. It is an ensemble of many decision trees, where each tree is trained on a random subset of the data and features, and the final prediction is made by averaging the outputs of all trees. The idea is that combining many imperfect trees reduces variance and leads to better generalization than any single tree.

While Random Forest gave a reasonable starting point, it did not perform well on this problem. The main reasons were its slow training speed on a large dataset, its difficulty handling highly imbalanced targets, and its limited ability to learn complex patterns compared to more modern alternatives. This result motivated a shift toward gradient boosting methods, which are generally better suited to tabular data tasks like this one.

2. Moving to Gradient Boosting: XGBoost, LightGBM, and CatBoost

The next step was exploring gradient boosting algorithms. Unlike Random Forest, which builds all trees in parallel, gradient boosting builds trees sequentially where each new tree tries to correct the mistakes made by the previous ones. This makes them much more powerful for structured tabular data.

i. XGBoost (Extreme Gradient Boosting)

XGBoost was the first gradient boosting model we tried. It is a widely used algorithm that introduced regularization techniques (L1/L2 penalties) to control overfitting. It offered a significant improvement over Random Forest, and confirmed that gradient boosting was the right direction for this problem.

ii. LightGBM (Light Gradient Boosting Machine)

LightGBM was the next model we explored. It was specifically designed for speed and efficiency on large datasets. Rather than splitting trees level by level like XGBoost, LightGBM uses a leaf-wise growth strategy that can find better splits faster. It also supports native handling of categorical features, meaning it can process columns like f7, f9, and f12 directly without needing manual one-hot encoding. LightGBM was trained with 5-fold stratified cross-validation, using early stopping to prevent overfitting, and produced strong AUC results.

iii. CatBoost (Categorical Boosting)

CatBoost is particularly well-suited for datasets with categorical variables. Its key strength is the way it handles categorical features internally through a technique called ordered target statistics, which avoids target leakage during encoding. Like LightGBM, CatBoost was trained using 5-fold stratified cross-validation with early stopping.

Feature and Hyperparameter Tuning

After establishing some baseline results with both LightGBM and CatBoost, more improvements were explored through two directions. First, the feature set was iterated on adding and removing engineered features, and assessing which combinations led to better OOF AUC. Second, hyperparameters were adjusted, including tree depth, regularization strength, subsampling ratios, and the number of boosting rounds. These changes were evaluated purely through OOF scores to avoid any form of test set leakage and overfitting.

Ensemble

The final step was combining CatBoost and LightGBM into a weighted ensemble. The motivation behind ensembling is that two well-performing models that make different types of errors can complement each other and their combined predictions tend to be more stable and accurate than either model alone. The optimal blend weight was found by searching over a range of CatBoost and LightGBM weight combinations from 0.0 to 1.0 in steps of 0.05, and selecting the combination that maximized the blended OOF AUC. The final submission used this optimized weighted average of the two models predicted probabilities.

Table 4 - Summary of model progression

Model	Notes
Random Forest	Baseline, low AUC and slow on large data
XGBoost	First gradient boosting attempt; notable improvement
LightGBM	Fast, strong AUC via 5-fold CV
CatBoost	Best categorical handling and strong AUC via 5-fold CV
CatBoost + LightGBM Ensemble	Final model and best overall AUC

Evaluation

Metric

The primary evaluation metric for this project is the Area Under the ROC Curve (AUC). AUC measures how well the model separates the two classes across all possible classification thresholds, making it far more informative than accuracy in heavily imbalanced settings like this one where 97.1% of samples belong to class 0. A model that simply predicts "no order" for every session would achieve 97% accuracy but an AUC of 0.5, which is no better than random guessing. AUC is therefore a much more honest measure of model quality here.

Validation Strategy

All models were evaluated using 5-fold stratified cross-validation. Stratification ensures that the imbalanced class ratio is preserved in every fold, so evaluation is consistent and not distorted by fold-level imbalance. Predictions were collected out-of-fold (OOF), meaning each sample was only scored by a model that had never seen it during training. This gives a reliable and unbiased estimate of generalization performance without ever touching the test set during model development.

Results

The table below shows the best OOF AUC achieved by each model during runtime:

Model	OOF AUC
CatBoost (5-fold)	0.971487
LightGBM (5-fold)	0.971949
Ensemble (0.40 CB + 0.60 LGB)	0.972290

Both individual models performed very closely to each other, with LightGBM having a slight improvement. The ensemble combined them at an optimal weight of 40% CatBoost and 60% LightGBM, found by grid searching over blend weights in steps of 0.05 and selecting the combination with the highest blended OOF AUC. This produced a small but consistent improvement over either model alone, which is the expected behavior when two strong models make partially different errors.

Kaggle Score

The final submission achieved a public leaderboard score of **0.97235**, which is closely in line with the OOF estimate of 0.97229. The close match between the two confirms that the validation strategy was sound and that the model generalizes well to unseen data without signs of overfitting.

Discussion

We faced three primary problems throughout this project.

1. Temporal Feature Representation

One of the earliest challenges was figuring out how to handle the three timestamp columns (f3, f4, f5). Initially they were dropped entirely since it was not obvious how to make use of raw datetime values in a machine learning model. After recognizing that time-based patterns existed in the data, the next attempt was converting them into session durations by taking differences between timestamps. This was better, but still left useful information on the table. We then attempted binning the hour of day into discrete categories, which captured some daily patterns but introduced some issues. The final approach was cyclical encoding using sine and cosine transformations of the hour and day of week. This was the best way to represent timestamp values since for example, hour 23 and hour 0 are numerically far apart but behaviorally adjacent, and cyclical encoding reflects that. This progression from dropping timestamps entirely to properly encoding their periodicity ended up being one of the more impactful improvements across the pipeline.

2. Hyperparameter Tuning and Model Switching

When working with XGBoost, improving the score through hyperparameter tuning proved difficult without a clear direction. Rather than continuing to experiment blindly, we switched to LightGBM and CatBoost, which handle categorical features natively and come with built-in early stopping. This made them much easier to work with and more effective for this specific problem.

3. Class Imbalance

The severe class imbalance (97.1% vs 2.9%) was a persistent challenge throughout the project. This is why AUC was used as the main evaluation metric. The gradient boosting models also benefit from their probabilistic outputs, since they are optimized directly on log-loss rather than accuracy, which forces them to learn meaningful probability estimates even for the minority class rather than simply defaulting to predicting class 0.

Conclusion

In this project, we successfully built a binary classifier to predict food delivery order placement, achieving a final AUC of 0.97235 on the Kaggle public leaderboard. The most impactful contributions were feature engineering particularly the cyclical encoding of temporal features and the treatment of structured missingness as a signal combined with an ensemble of LightGBM and CatBoost evaluated through out-of-fold cross-validation. Overall, the project demonstrated how combining feature design, model tuning, and ensemble learning can significantly improve predictive performance in an imbalanced classification problem.